P5 · DYNAMIC WORKFLOW

Dynamic Workflow

품질 검증 루프 — Claude가 자신의 하네스를 즉석에서 쓴다

2026년 5월 28일 Claude Code에 도입. 평문 요청을 받아 오케스트레이션 스크립트를 직접 작성하고,
수십~수백 서브에이전트를 병렬로 돌린 뒤 결과가 사람에게 닿기 전에 스스로 검증한다.

이 파트의 세 가지 메시지

오케스트레이션이 개발자의 결정에서 모델의 결정으로 옮겨 왔다

1

Claude가 하네스를 쓴다

작업을 보고 JS 오케스트레이션 스크립트를 직접 작성·조율한다.

2

조율은 0토큰

조율 자체는 스크립트라 모델 토큰을 쓰지 않는다.

3

가치는 속도보다 신뢰

반박·교차검증 같은 반복 가능한 품질 패턴이 신뢰를 만든다.

새로운 것은 프리미티브가 아니라, 사람이 조립할 양을 줄였다는 점이다

세 조건이 겹칠 때 빛난다

Breadth · Verification · Scale

Breadth · 폭

- 확인할 갈래가 많을 때
- 여러 출처·모듈·콜사이트

Verification · 검증

- 결과를 믿기 전에
- 반박·교차검증이 필요할 때

Scale · 규모

- 단계마다 깨끗한 컨텍스트로
- 대량 병렬화할 때

Claude가 하네스를 즉석에서 쓴다

스크립트로 조율한다

- 작업을 보고 JS 오케스트레이션 프로그램을 작성
- 수십~수백 서브에이전트를 병렬 실행
- 결과가 사람에게 닿기 전에 스스로 검증
- 사람은 JavaScript를 손으로 쓰지 않는다

조율은 0토큰

- 조율 자체는 스크립트라 모델 토큰을 쓰지 않음
- 계획을 코드로 굳혀 반복 가능한 패턴 적용
- 실행 전 계획을 보여주고 허락을 구한다
- 병렬 에이전트가 많아 토큰 사용량이 높다고 경고

빌딩 블록은 단 하나, 에이전트

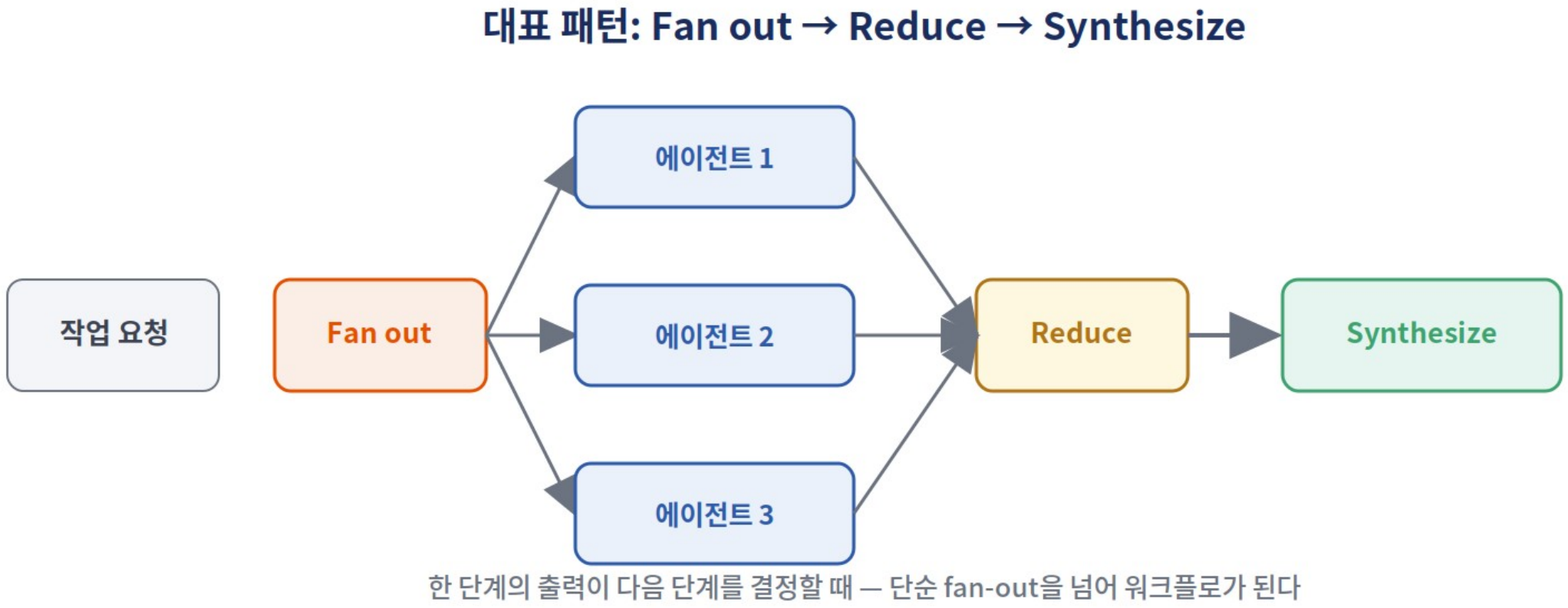
에이전트로 두 종류의 프로그램을 짠다

프리미티브	역할
agent()	하나의 에이전트 — 한 작업을 수행
parallel()	여러 에이전트를 동시에 실행 (펼침)
pipeline()	여러 에이전트를 순서대로 실행 (사슬)
workflow()	위를 묶어 하나의 재사용 가능한 하네스로

parallel()로 넓게 펼치고, pipeline()으로 단계를 잇고, 중간 가공은 평범한 JavaScript로.

Fan out → Reduce → Synthesize

넓게 펼치고, 모으고, 종합한다



품질을 높이는 패턴

누구도 자기 숙제를 자기가 채점하지 않는다

패턴	방식	효과
Adversarial Verify	한 에이전트가 다른 에이전트의 주장을 반박	환각·근거 없는 주장 제거
Diverse Lenses	같은 문제를 여러 관점으로 동시 검토	사각지대 축소
Judge Panel	여러 심판이 결과를 평가·투표	편향된 단일 판정 방지
Loop-until-dry	더 나올 게 없을 때까지 반복	누락 최소화

검증은 별도 에이전트가, 가능하면 별도 컨텍스트 창에서 수행한다.

같은 기능을 켜는 세 가지 길

바로 쓰는 번들 · 자동 판단 · 직접 설계



/deep-research 해부

검색을 분해해 폭을 넓히고, 자기 주장을 반박해 검증한다



언제 쓰고, 무엇을 피하나

언제 무엇을

- Subagent — 1~2개의 격리된 조사
- Skill — 노하우를 재사용하고 싶을 때
- Agent Team — 고정된 팀이 필요할 때
- Workflow — 조율 자체가 반복 가능해야 할 때

피해야 할 안티패턴

- 단순 작업의 과복잡화
- 검증 없는 대규모 병렬
- 1000행을 한 프롬프트에 (토너먼트로 분해)

두 개의 토론 질문

Q1 fan-out과 워크플로의 경계

'한 번 펼쳐 모으면 끝나는 일'과 '한 단계 결과가 다음 단계를 바꾸는 일'을 하나씩 꼽아 보자. 후자에 어떤 검증 패턴을 끼워 넣겠는가?

Q2 자기 채점 금지

최근 결과물이 그럴듯했지만 틀렸던 사례를 떠올려 보자. Adversarial Verify나 Judge Panel을 어느 지점에 넣었다면 그 오류를 잡을 수 있었겠는가?

한 줄 정리

"Claude가 하네스를 즉석에서 쓰고, 반박 가능한 품질 패턴이 신뢰를 만든다."

다음: P6 — 실전 응용 (개발을 넘어)

여기까지가 하네스를 만들고 실행하는 기술이다.